

Manual de Engenharia e Arquitetura de Backend

Versão 1.0 — Guia de Referência Corporativa para Microsserviços e APIs

1. Visão Geral da Arquitetura e Comunicação Híbrida

O ecossistema de backend é composto por microsserviços distribuídos desenvolvidos primariamente em Python (FastAPI) e Go, orquestrados em clusters Kubernetes (K8s). A comunicação entre os serviços opera no modelo híbrido: síncrona via REST/gRPC para consultas transacionais de baixa latência e assíncrona orientada a eventos (Event-Driven) para processamento em background.

1.1 API Gateway Kong e Padrões de Segurança

O Kong API Gateway é a porta de entrada única do sistema, encarregado de terminação SSL, CORS, rate limiting e autenticação. O Rate Limiting padrão estabelece 100 requisições por minuto por IP para endpoints públicos e 1.000 requisições por minuto para clientes autenticados via `client_id`.

Endpoints privados exigem autenticação via JSON Web Token (JWT) no cabeçalho `Authorization: Bearer <TOKEN>`. As claims obrigatórias do payload são: `sub` (identificador único UUID do usuário), `roles` (lista de permissões, ex: `[developer, admin, viewer]`) e `exp` (timestamp de expiração com validade máxima de 60 minutos).

1.2 Mensageria e Eventos com Apache Kafka

A publicação de eventos no Apache Kafka deve seguir rigidamente a especificação CloudEvents v1.0 em JSON. A convenção de nomenclatura para tópicos segue o formato: `<ambiente>.<domínio>.<entidade>.<evento>` (exemplo: `prod.checkout.pedidos.criado`). Todo tópico de produção deve possuir no mínimo 3 partições ativas e retenção padrão de 7 dias.

2. Diretrizes de Design de APIs RESTful e Tratamento de Erros

Todas as APIs devem aderir à especificação OpenAPI 3.1. Recursos devem ser nomeados no plural (ex: /v1/pedidos, /v1/usuarios). Operações de criação utilizam POST retornando status 201 Created com cabeçalho Location. Atualizações parciais utilizam PATCH e deleções retornam 204 No Content.

2.1 Padrão RFC 7807 para Respostas de Erro (Problem Details)

Todas as respostas de erro (HTTP 4xx e 5xx) devem retornar obrigatoriamente a estrutura padronizada da RFC 7807:

```
{ "type": "https://api.borus.com/errors/not-found", "title": "Recurso Nao Encontrado", "status": 404,
  "detail": "O pedido com identificador ord_12345 nao foi localizado na base de dados.", "instance":
  "/v1/pedidos/ord_12345" }
```

2.2 Paginação, Ordenação e Idempotência

Coleções paginadas utilizam os parâmetros page (iniciando em 1), page_size (padrão: 20, limite máximo permitido: 100) e sort (prefixo - para ordenação decrescente).

Operações críticas de escrita (como pagamentos e checkout) exigem o cabeçalho Idempotency-Key com UUID v4 único. O resultado da operação é cacheado no Redis por 24 horas para evitar cobranças ou criações duplicadas em caso de retentativas de rede.

3. Persistência de Dados e Estratégia de Cache

O banco relacional corporativo é o PostgreSQL 16, operando no padrão Database-per-Service. Toda conexão deve passar pelo PgBouncer em modo transaction pooling para otimizar o consumo de conexões e prevenir saturação.

3.1 Padrões de Modelagem no PostgreSQL 16

1. Chaves Primárias: Utilizar UUID v7 ordenável por tempo ou BIGSERIAL para tabelas transacionais de alto volume. 2. Índices: Criar índices B-Tree para colunas de junção (foreign keys) e filtros frequentes. Utilizar índices parciais para flags de status ativo/inativo.

3.2 Estratégia de Caching com Redis 7 (Cache-Aside)

O Redis 7 é utilizado como camada de cache em memória de alta performance (latência de leitura < 2ms). O padrão adotado é Cache-Aside (Lazy Loading): a aplicação consulta o Redis; em caso de cache miss, consulta o PostgreSQL e popula o Redis com TTL.

Tipo de Dado	Padrão de Chave Redis	TTL Recomendado
Perfil de Usuário	user:profile:<uuid>	30 minutos (1800s)
Catálogo de Itens	catalog:item:<sku>	2 horas (7200s)
Sessão / Token JWT	auth:session:<session_id>	15 minutos (900s)
Chave de Idempotência	idempotency:<uuid>	24 horas (86400s)

Invalidação de Cache: Toda operação de escrita (POST, PUT, DELETE) deve disparar invalidação síncrona imediata da chave correspondente no Redis antes de responder ao cliente (Write-Through Invalidation).